

RoleRAG Interviewer

Detailed project explanation covering system purpose, architecture, backend and frontend design, RAG pipeline, data flow, APIs, deployment, trade-offs, limitations, and suggested improvements.

PROJECT TYPE

Full-stack RAG interview platform

CORE BACKEND

FastAPI + SQLAlchemy + ChromaDB

FRONTEND

React + Vite + Axios

1. Project Overview

RoleRAG Interviewer is a role-aware technical interview application. It starts with a candidate resume, extracts a structured profile, retrieves role-specific knowledge-base content, generates grounded interview questions, evaluates answers, and stores the full session trace.

The project is stronger than a single prompt-based demo because it supports a complete interview lifecycle: input collection, retrieval, adaptive questioning, evaluation, session persistence, summary generation, and reset.

Resume-aware

Role-specific RAG

Adaptive follow-up

Turn-level persistence

Summary generation

2. Primary Objective

The project solves a simple but important problem: generic interview systems ask shallow questions because they do not understand the candidate or the target role. This system improves relevance by combining two signals:

1. The candidate's actual resume, parsed into a structured profile.
2. A role-specific knowledge base retrieved through embeddings and vector search.

That combination lets the interviewer ask questions that are both personalized and grounded.

3. Supported Roles

- AI/ML Engineer
- Backend Engineer
- Data Science / Applied ML

Each role maps to its own Chroma collection. This avoids cross-role noise and keeps retrieval focused.

4. High-Level Architecture

Frontend: Candidate uploads resume, selects role, answers questions, views evaluations and final summary.

API Layer: FastAPI validates input, orchestrates session creation, answer submission, retrieval, and response formatting.

Resume Service: Extracts text from PDF/TXT and creates a structured profile using OpenAI when available, with deterministic fallback parsing.

RAG Service: Ingests documents, chunks them, embeds them, stores vectors, builds queries, retrieves relevant context, and deduplicates results.

LLM Service: Generates questions, evaluates answers, and writes a final summary. It also includes fallback logic when no OpenAI API key is configured.

Persistence: SQLAlchemy stores interview sessions and turn-level trace data in SQLite.

5. Project Structure

Path	Responsibility
backend/app/main.py	FastAPI app initialization, CORS setup, router registration, DB startup, health endpoint.
backend/app/api/interview.py	Interview lifecycle endpoints including KB ingestion, start, answer, summary retrieval, and reset.
backend/app/services/resume.py	Resume extraction and structured profile parsing.
backend/app/services/rag.py	Document ingestion, chunking, embedding, collection management, query building, and retrieval.
backend/app/services/llm.py	Question generation, answer evaluation, session summarization, and fallback behavior.
backend/app/models/db.py	SQLAlchemy models for interview sessions and turns.
backend/app/schemas/interview.py	Pydantic response and request schema definitions.
frontend/src/App.jsx	Main client state orchestration and API interaction.
frontend/src/components/*	Candidate entry, question UI, progress stepper, evaluation cards, turn log, summary, and error banner.
docker-compose.yml	Runs backend and frontend together for local containerized execution.

6. End-to-End Request Flow

Interview Start

1. User uploads a PDF or TXT resume and selects a role.
2. Backend saves the upload under backend/data/uploads.
3. Resume text is extracted.
4. The extracted text is converted into a profile dictionary.
5. RAG queries are generated from role, skills, projects, and domain hints.
6. Matching context is retrieved from the role-specific Chroma collection.
7. A question is generated using resume profile plus retrieved context.
8. A new DB session row and the first interview turn are stored.

Answer Submission

1. The current pending interview turn is loaded from the database.
2. The answer is validated and evaluated.
3. The turn is updated with the answer and evaluation payload.
4. If max turns are not reached, a new retrieval step runs using the latest answer as extra context.
5. The next question is generated and stored as a fresh pending turn.
6. If max turns are reached, a final session summary is generated and saved.

7. Backend Design

App Initialization

The FastAPI app reads settings from environment variables, sets CORS, registers the interview router, and creates database tables on startup.

Configuration

Config values include app name, SQLite path, Chroma directory, upload directory, KB directory, embedding model name, OpenAI model, allowed origins, and max turns.

Default OpenAI model: gpt-4o-mini

Default embedding model: sentence-transformers/all-MiniLM-L6-v2

API Endpoints

Endpoint	Purpose
GET /health	Simple readiness check.
POST /api/kb/ingest	Indexes role-specific documents into Chroma.
POST /api/interviews/start	Starts a session from role + resume upload.
POST /api/interviews/{session_id}/answer	Stores answer, evaluates it, and returns the next question or done state.
GET /api/interviews/{session_id}	Returns full session summary including turns and evaluations.
POST /api/interviews/{session_id}/reset	Deletes prior turns and regenerates a fresh first question for the same session.

8. Database and Data Model

InterviewSession

- id
- candidate_name
- role
- resume_text
- extracted_profile as JSON
- summary
- created_at

InterviewTurn

- id
- session_id
- question
- answer
- retrieved_context as JSON
- evaluation as JSON
- created_at

This structure is important because it preserves the complete evidence trail for each interview step. The system can always show which sources grounded a question and how an answer was evaluated.

9. Resume Parsing Logic

The resume parser first extracts raw text from a PDF or TXT file. It then tries to build a structured candidate profile.

LLM-first Path

- Uses OpenAI if OPENAI_API_KEY is present.
- Requests JSON with name, contacts, skills, years of experience, domains, projects, seniority, and summary.

Fallback Path

- Normalizes lines and detects common resume sections.
- Extracts possible name, emails, phone numbers, years of experience, project bullets, and skill categories.
- Infers domains such as backend, AI/ML, frontend, and cloud/devops from keyword presence.

Strength: the project still works without an API key, which makes the demo much more reliable.

10. RAG Pipeline

Ingestion

1. Load role-specific PDFs, TXT files, and Markdown files from backend/data/kb_docs.
2. Extract text from each file.
3. Split text into chunks of about 950 words with 160-word overlap.
4. Embed chunks using `all-MiniLM-L6-v2`.
5. Store them in persistent Chroma collections keyed by role.

Retrieval

1. Build multiple queries from role, skills, projects, domains, experience level, and the previous answer.
2. Query the correct collection for each prompt.
3. Collect document text, metadata, and vector distance.
4. Deduplicate chunks by source and chunk id.
5. Return the top few unique context blocks.

The system stores metadata like source document name, role, and chunk number so the UI can display question grounding.

11. LLM Service Behavior

Question Generation

Question prompts include candidate role, experience level, skills, projects, resume highlights, retrieved context, prior questions, latest answer, and current turn number. The prompt specifically instructs the model to ask one practical, non-generic, non-repeated question.

Fallback Questions

If the OpenAI client is unavailable or fails, the system uses deterministic templates. These templates still try to reference project names, role-specific concepts, trade-offs, debugging, monitoring, and deployment concerns.

Answer Evaluation

With OpenAI enabled, the answer is scored and summarized as JSON. Without it, the fallback evaluator checks depth, examples, trade-offs, and debugging signals to produce a score plus strengths, gaps, and a suggested follow-up.

Final Summary

At the end of the interview, the system creates a narrative summary of overall signal, strengths, risks, follow-up areas, and hiring recommendation. There is also a fallback summary mode when the LLM is unavailable.

12. Frontend Design

The frontend is a single-page React interface built around interview state rather than routing.

Component	Role in UI
CandidateEntry	Role selector and resume upload form for starting the session.
InterviewQuestion	Current question, source grounding, answer box, and submit button.
Stepper	Turn progress visual for the interview lifecycle.
TurnLog	Shows extracted profile, source documents, previous questions, answers, and evaluation cards.
EvaluationCard	Displays score, strengths, gaps, and follow-up advice.
SummaryView	Final narrative summary and average score after all turns are completed.
ErrorBanner	Displays backend validation or runtime errors to the user.

The frontend also computes a running average score from completed turns and displays it while the interview progresses.

13. Deployment and Runtime

Local Development

- Backend served with `uvicorn app.main:app --reload`
- Frontend served with `npm run dev`

Docker

- Backend container uses Python 3.11 slim.
- Frontend container uses Node 20 Alpine.
- `docker-compose.yml` exposes backend on port 8000 and frontend on port 5173.
- Backend data is mounted so uploads and Chroma persistence survive container restarts.

14. Strengths of the Current Implementation

- Clear separation between API, services, schemas, models, and UI components.
- Useful fallback behavior without depending completely on external APIs.
- Role-specific retrieval strategy is simple and easy to explain.
- Turn-level traceability makes the project auditable and demo-friendly.
- Reset flow is practical for repeated demo recordings.
- The system returns grounding sources to the frontend, which improves explainability.

15. Current Limitations and Risks

- Database is SQLite, which is fine for local demos but limited for production concurrency.
- Knowledge-base ingestion runs inline, which may be slow for large document sets.
- No authentication, user accounts, or admin workflows are implemented.
- There is no advanced observability for retrieval quality, latency, or prompt usage.
- Document freshness and embedding invalidation are not handled yet.
- Fallback evaluation is heuristic, so it is less nuanced than model-based evaluation.

The project is strong as a prototype and assignment submission, but production-readiness would require better security, async/background processing, more monitoring, and a more robust database layer.

16. Suggested Future Improvements

- Replace SQLite with PostgreSQL.
- Add async or background ingestion for large PDFs.
- Support richer file types and document version tracking.
- Add retrieval metrics and prompt tracing.
- Introduce role rubrics and interviewer configuration panels.
- Add downloadable candidate reports.
- Add admin dashboards and authentication.

17. Final Summary

RoleRAG Interviewer is a solid full-stack RAG application because it combines retrieval, personalization, adaptive questioning, structured evaluation, and persistent session storage into one coherent workflow.

The strongest story to tell about this project is not just that it uses RAG, but that it uses RAG in a way that improves interview quality: resume-aware inputs, role-specific grounding, adaptive follow-up, and a retained evidence trail.